课程：并行程序设计

姓名：张建夫

学号：10235101477

1. 问题的描述

1）编程语言必须使用 IEEE Posix Thread 库与 C 语言实现。需要采用多线程实现。哲学家的刀叉由于是共享变量，所以需要作为临界区用互斥变量、信号量等机制实现互斥。 本实验规定采用锁机制与信号量两种方式实现互斥。
2）本次哲学家问题主要想考察大家对于死锁形成机制的理解。希望大家实现的一个能够导致死锁的 primitive 哲学家问题。程序的输入可以指定哲学家个数。 每个哲学家分为 3 个状态｛thinking，trying，eating｝，分别代表思考，尝试获得叉子，吃面三个状态。规定 thinking 于 eating 的时间在区间［0.01, 0.1］秒之间。所谓死锁就是大家都处于 trying 的状态。请给出相应的状态输出序列。
3）为了避免死锁，有许多的解决方法。请你列举 2 种方法，并用程序实现，输出状态序列。
4）请尝试使用锁的机制，使得哲学家按照其编号进行吃面，如果可以请给出代码，如果不可以请说明理由。

2. 概要设计

核心设计：

1）两种解决方法和会导致死锁的正常方法通过一个枚举类区分，资源（筷子）使用锁来表示，一个资源一把锁，上锁说明资源被占用。在线程函数（代表哲学家）中使用 switch 来处理不同模式

2）第一个解决方法为使用信号量让一次最多只有 n - 1 个哲学家进入临界区，这样必有一个哲学家能打破“死锁状态”，进而所有哲学家都能吃上饭(模式代码为 -sema)

3）另一个解决方案是让所有哲学家按顺序吃饭，使用一个 flag 变量,一个锁以及一个条件变量实现，flag 表示当前应该吃饭的哲学家编号，其他编号的哲学家会将自己 sleep 并释放锁，直到对应编号的哲学家拿到锁(模式代码为 -order)

其他设计：

1）错误处理，限制输入只能按照如下方式：

./philosopher [-mode(仅能为三个值，-normal，-sema，-order)] -n [哲学家的

个数(大于 1)]

否则程序退出。

2）自定义睡觉函数，让睡觉时间固定为[0.01, 0.1]秒。

3. 详细介绍避免死锁的策略

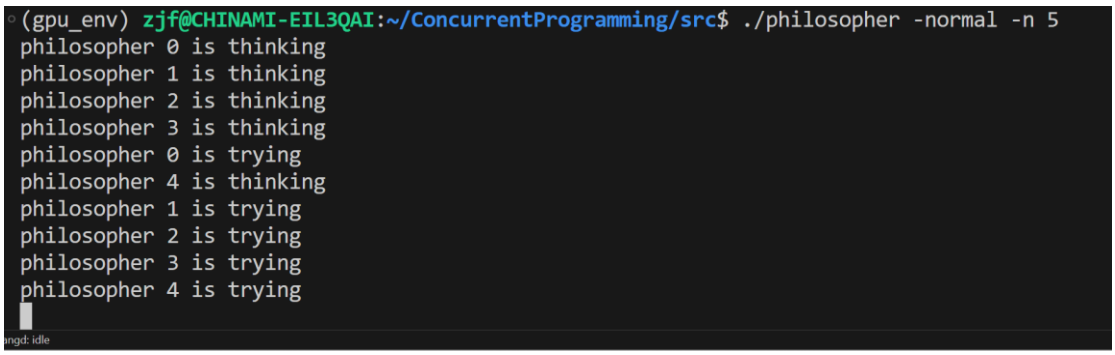
1）第一种方案：使用信号量让一次最多只有 n - 1 个哲学家进入临界区，

由于总有一个哲学家能拿到两个筷子（死锁的条件是哲学家都拿左边的或右边的筷子但是由于少了一个人，最左或最右就会至少一个人有两个筷子），故必有哲学家能吃完饭，这样所有哲学家就都能吃完饭。

2）第二种方案：让所有哲学家按顺序吃饭，使用一个 flag 变量,一个锁以及一个条件变量实现，flag 表示当前应该吃饭的哲学家编号，其他编号的哲学家会将自己 sleep 并释放锁，直到对应编号的哲学家拿到锁，对应编号的哲学家拿到锁吃完饭后就广播所有人，其他人检查自己的条件是否符合，符合就拿锁吃饭，否则继续睡觉。

4. 实验结果

实验结果主要以哲学家数量为 5 进行展示。



Normal 模式下五个线程都在 trying，死锁了。

```
(gpu_env) zjf@CHINAMI-EIL3QAI:~/ConcurrentProgramming/src$ ./philosopher -sema -n 5
philosopher 0 is thinking
philosopher 1 is thinking
philosopher 2 is thinking
philosopher 1 is trying
philosopher 3 is thinking
philosopher 4 is thinking
philosopher 2 is trying
philosopher 0 is trying
philosopher 3 is trying
philosopher 4 is trying
philosopher 3 is eating
philosopher 2 is eating
philosopher 1 is eating
philosopher 0 is eating
philosopher 4 is eating
(gpu_env) zjf@CHINAMI-EIL3QAI:~/ConcurrentProgramming/src$ ./philosopher -sema -n 5
philosopher 0 is thinking
philosopher 1 is thinking
philosopher 2 is thinking
philosopher 0 is trying
philosopher 1 is trying
philosopher 3 is thinking
philosopher 4 is thinking
philosopher 1 is eating
philosopher 2 is trying
philosopher 3 is trying
philosopher 4 is trying
philosopher 0 is eating
philosopher 4 is eating
philosopher 3 is eating
philosopher 2 is eating
(gpu_env) zjf@CHINAMI-EIL3QAI:~/ConcurrentProgramming/src$
```

Sema 模式下无死锁，吃饭顺序随机。

```
(gpu_env) zjf@CHINAMI-EIL3QAI:~/ConcurrentProgramming/src$ ./philosopher -order -n 5
philosopher 0 is thinking
philosopher 0 is trying
philosopher 1 is thinking
philosopher 2 is thinking
philosopher 3 is thinking
philosopher 0 is eating
philosopher 1 is trying
philosopher 4 is thinking
philosopher 2 is trying
philosopher 3 is trying
philosopher 4 is trying
philosopher 1 is eating
philosopher 2 is eating
philosopher 3 is eating
philosopher 4 is eating
(gpu_env) zjf@CHINAMI-EIL3QAI:~/ConcurrentProgramming/src$ ./philosopher -order -n 5
philosopher 0 is thinking
philosopher 1 is thinking
philosopher 2 is thinking
philosopher 1 is trying
philosopher 0 is trying
philosopher 3 is thinking
philosopher 4 is thinking
philosopher 0 is eating
philosopher 2 is trying
philosopher 3 is trying
philosopher 4 is trying
philosopher 1 is eating
philosopher 2 is eating
philosopher 3 is eating
philosopher 4 is eating
(gpu_env) zjf@CHINAMI-EIL3QAI:~/ConcurrentProgramming/src$
```

Order 模式下所有哲学家的吃饭顺序一致（eating 按照升顺）

5. 分析

1）死锁的原因：

循环等待，所有哲学家都只拿了自己左手边或右手边的筷子，导致所有资源被占用，而程序执行的条件又要求那另一只筷子，自然所有人都动不了

2）解决方案的思路：

思路一：减少哲学家或者加一支筷子，故可以使用信号量限制同一时间哲学家吃饭的个数，保证高并发的同时却不会死锁。

思路二：使用条件变量控制哲学家的吃饭顺序，条件变量使各个编号的哲学家仅有在条件满足的时候才能吃饭，这也解答了第五问的问题，条件变量可以使用锁加等待队列实现，故使用锁可以实现哲学家按照编号吃面，代码如下：

```
case order :
    pthread_mutex_lock(mutex: &order_mutex);
    while (flag != my_rank) {
        pthread_cond_wait(cond: &cond, mutex: &order_mutex);
    }
    pthread_mutex_lock(mutex: &mutexs[my_rank]);
    // 现代处理器速度太快，两次加锁之间需要小睡一下，不然无法测试实现是否正确
    my_sleep();
    pthread_mutex_lock(mutex: &mutexs[(my_rank + 1) % thread_count]);
    // 两双筷子都拿到了，开吃
    printf(format: "philosopher %d is eating\n", my_rank);
    my_sleep();
    pthread_mutex_unlock(mutex: &mutexs[my_rank]);
    pthread_mutex_unlock(mutex: &mutexs[(my_rank + 1) % thread_count]);
    flag++;
    pthread_cond_broadcast(cond: &cond);
    pthread_mutex_unlock(mutex: &order_mutex);
    break;
```